Functional Design

Jens Bouma
Nicolas Llop Prieto
Husna Keskin
Ben Wierenga
Matz Leeflang

Version Control

Version	Date	Description
0.1	01-10-2024	Initial setup FD
0.2	05-10-2024	Setting up principles
0.3	10-10-2024	Added domain analysis situation and goal
0.4	13-10-2024	Added further structure to domain analysis and use cases in chapters.
0.5	20-10-2024	Use cases for prototype
0.6	22-10-2024	Use cases for dashboard, Keuzewijzer and general added for first sprint.
0.7	28-11-2024	Use cases for password reset and managing routes and cohorts
0.8	06-12-2024	Use cases for free choice and retake cluster added
0.9	11-12-2024	Updated introduction and updated figures overview
0.10	12-12-2024	Use cases for managing clusters and users added
0.11	18-12-2024	Changes after feedback: - Adding traceability numbers - Improved titles, acceptance criteria, use case description
0.12	03-01-2025	Language checks

Table of Contents

1. Introduction 1

2. Domain analysis 2

2.1. Situation and goal 2

2.2. Domain model 2

2.3. Actors 3

2.4. Use Case Diagram 3

3. Principles 4

4. User Stories 5

4.1. Prototype 5

4.1.1. Deploying 5

4.2. Keuzewijzer 6

4.2.1. Show graduate profiles 6

4.2.2. Select study year 8

4.2.3. Select graduate profile 9

4.2.4. Select study route 10

4.2.5. View Cluster Information 12

4.2.6. Insert free choice 13

4.2.7. Retake cluster 14

4.3. Dashboard 16

4.3.1. Login dashboard 16

4.3.2. Reset password 17

4.3.3. Manage graduate profiles 18

4.3.4. Manage cohorts 19

4.3.5. Manage clusters 19

4.3.6. Manage users 21

4.4. General 22

4.4.1. Implementing a CI/CD pipeline 22

Figures 23

1. Introduction

The Keuzewijzer allows students to select and complete clusters, which are essential components that fulfill the requirements of various graduation profiles. These clusters enable students to expand and deepen their knowledge within their chosen fields, providing a comprehensive academic experience.

In addition to the structured curriculum, the Keuzewijzer empowers students by allowing them to personalize their studies through free choice electives, including additional clusters or internships that align with their career aspirations and personal interests.

This document further elaborates on the tool's functionalities, outlining in detail the use cases, domain model and design models that are essential for understanding and using the Keuzewijzer effectively.

For the traceability of the requirements throughout the process, the following explains how this will be achieved. Data carriers in the documentation will be provided with an identifier.

Identifier	Description
P<#>	Principle
FR<#>	Functional Requirement
NFR<#>	Non-Functional Requirement

2. Domain analysis

This chapter defines the domain of the Keuzewijzer. The domain knowledge was gathered through interviews with the product owner, students, and student coaches, the key stakeholders. These interviews have been documented in the interview notes. The domain aligns with the collected requirements. To provide a clear understanding of the project's domain, both a domain model and a use case diagram are included.

2.1. Situation and goal

All first-year students who started the HBO-ICT program at Windesheim this academic year cannot use the current Keuzewijzer. This is because the curriculum of the new HBO-ICT program has changed compared to previous years. As a result, these students, unlike those before them, cannot use the existing Keuzewijzer to select a study path through to graduation. The goal of the new Keuzewijzer is to address this by providing first-year and future students with a tool that allows them to easily map out their study plan by selecting clusters and visualizing their journey through the HBO-ICT program.

2.2. Domain model

The domain model of the Keuzewijzer includes clusters that can belong to multiple graduate profiles and be followed in the first, second, or both semesters of a study year. Clusters can also be free choices, defining the student's study route.

Clusters have levels that separate them. For instance, a level 2 cluster can only be followed after completing two level 1 clusters. They also have links that open Sharenet pages with cluster information.

Cohorts represent the year a student starts their studies.

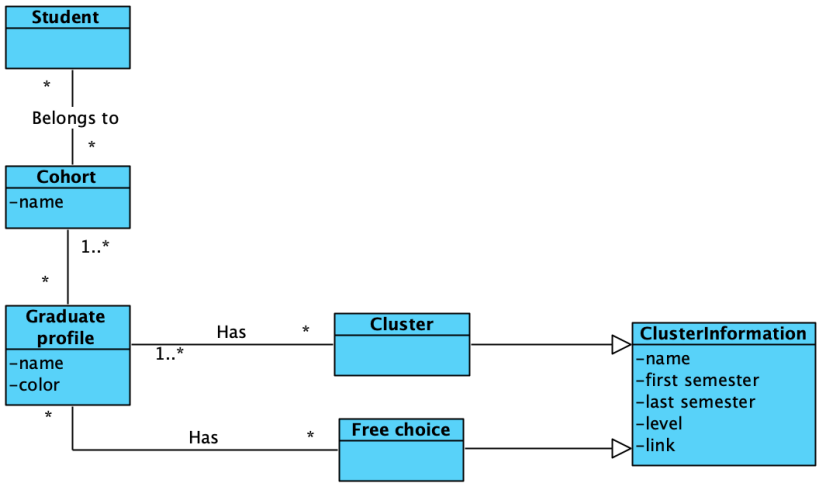


Figure 1 Domain model

2.3. Actors

Name	Actor Type	Description
Student/Student Advisor	Primary	The user who interacts with the tool to follow study plans.
Maintainer	Primary	The user who can edit information from the Keuzewijzer in the dashboard.
Product Owner	Primary	Responsible for the project and the continuation of the project.

2.4. Use Case Diagram

The use case diagram describes the use cases of the Keuzewijzer and the dashboard.

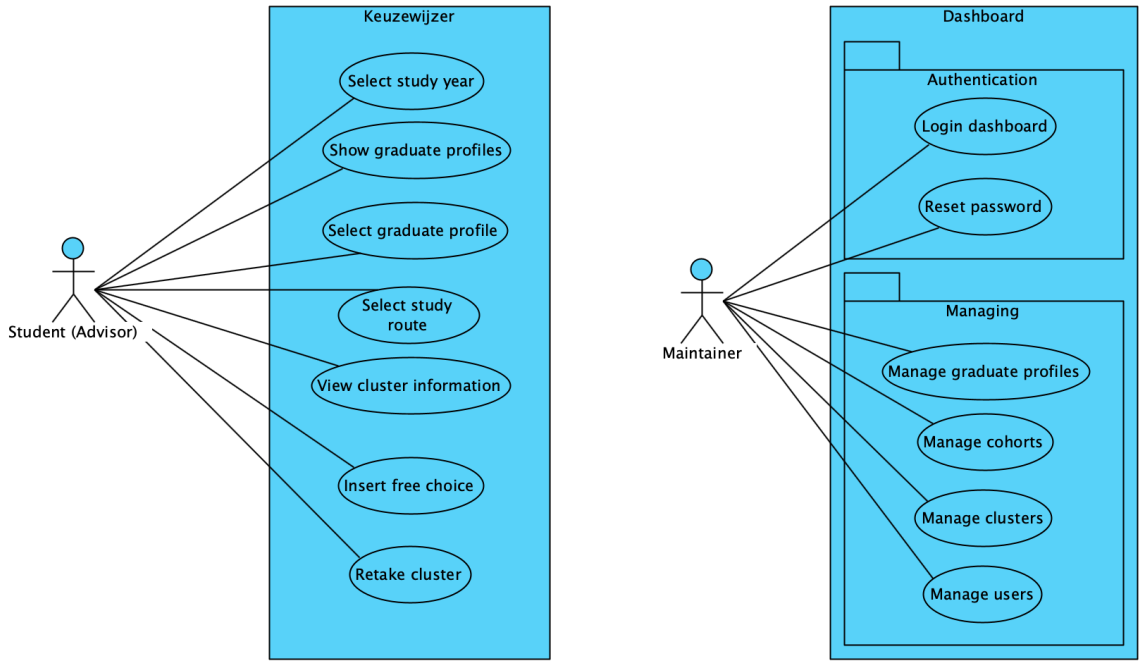


Figure 2 Use Case Diagram of Keuzewijzer and Dashboard

3. Principles

Principles are acceptance criteria that applicable for all the use cases, if necessary.

#	Description	MoSCoW
P1	Responsive design	Must
P2	Frontend has been tested by Cypress	Must
P3	Backend has been tested by Unit Tests	Must
P4	The system must comply with relevant regulations (e.g., GDPR, accessibility standards), ensuring that all functionalities adhere to legal requirements.	Must
P5	All features and functionalities should prioritize the end user's experience, focusing on ease of use, accessibility, and intuitive interaction.	Must
P6	The design, terminology, and interactions should be consistent across all parts of the project, ensuring users can navigate and use the system without confusion.	Should
P7	Security best practices must be adhered to, ensuring data protection, authentication, and authorization standards across all features.	Must
-	Code and documentation should be clear and well-organized to facilitate easy maintenance, updates, and debugging.	Should

4. User Stories

This chapter describes the realized user stories of the Keuzewijzer and the dashboard of the Keuzewijzer.

4.1. Prototype

In this chapter the deployment use case of the prototype has been described. This is the first stage of the project.

4.1.1. Deploying

As a Product Owner, I want to deploy the prototype to a live or staging environment so that stakeholders and users can access, test, and provide feedback on the functionality and usability of the prototype.

4.1.1.1. Acceptance criteria

#	Description	MoSCoW
FR0.1	The prototype is successfully deployed to a live or staging environment.	Must
FR0.2	The prototype is accessible via a URL or other access points.	Must
FR0.3	The deployed prototype works on multiple devices	Should
FR0.4	Deployment should include any necessary documentation or instructions for testing the prototype.	Should

4.1.1.2. Use case description

Use Case	Deploying a prototype
Actor	Product Owner
Precondition	The prototype has been developed with core features and is ready for user testing.
Main Flow	1. The project team prepares the prototype for deployment, ensuring all key features are functional. 2. The team deploys the prototype to a test environment accessible via a secure link or URL. 3. The deployment is announced to stakeholders and test users, providing them with access instructions and any relevant documentation. 4. Stakeholders and test users access the prototype and interact with its features.
Postcondition	The prototype is live in a test environment and accessible to stakeholders and test users for feedback and testing.

Alternative Flow	(3a) If deployment issues arise (e.g., server errors or access issues), the project team troubleshoots and redeploys as needed before notifying stakeholders.

4.2. Keuzewijzer

In this chapter all the user stories that belong to the Keuzewijzer itself will be described.

4.2.1. Show graduate profiles

As a student, I want to see the available graduate profiles, so that I can see which path I have to choose to graduate with that profile.

4.2.1.1. Acceptance criteria

#	Description	MoSCoW
FR1.1	The system shows the available graduate profiles.	Must
FR1.2	The system should ensure the list is updated with current profiles based on the program curriculum.	Must
FR1.3	The system must keep showing the graduate profiles after one has been selected.	Must
NFR1.1	A graduate profile should have its own color, so it can be visually separated.	Must

4.2.1.2. Use case description

Use Case	Show Graduate Profiles
Actor	Student
Precondition	The student has accessed the Keuzewijzer webpage.
Main Flow	1. The student selects a study year. 2. The system displays a list of available graduate profiles. 3. The student selects a graduate profile.
Postcondition	The student still sees the graduation profiles while seeing the path of the selected graduation profile.
Alternative Flow	(1a) If the selected study year doesn't belong to the new curriculum, the system navigates to the old Keuzewijzer. (2a) If no profiles are available (due to a system error or update), the system displays a message and suggests checking back later or contacting an advisor.

4.2.1.3. Designs



Figure 3 Mobile design Graduate profiles.

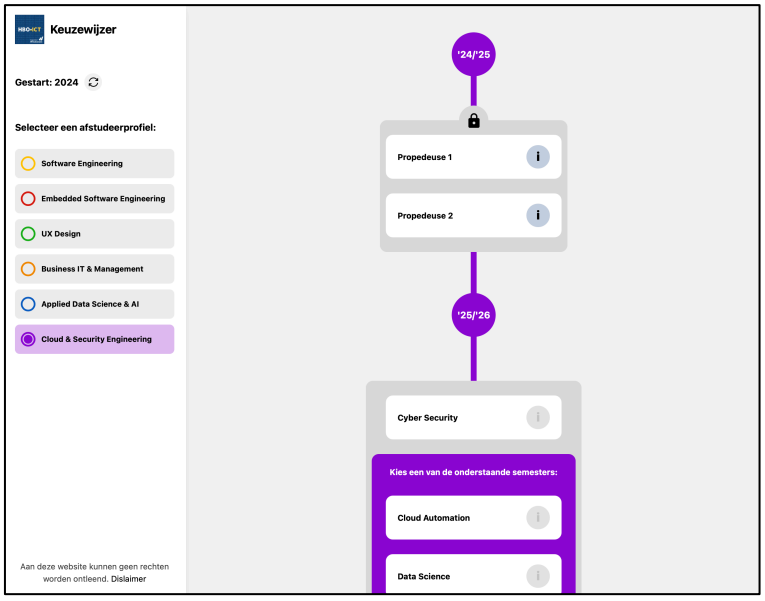


Figure 4 Tablet/Desktop design of location graduate profiles

4.2.2. Select study year

As a student, I want to select a study year in the Keuzewijzer so that I can see an overview of paths available to me.

4.2.2.1. Acceptance criteria

#	Description	MoSCoW
FR2.1	When opening the webpage for the first time it should show a list of years that can be selected when a student started their study.	Must
FR2.2	The current selected study year can be changed	Must
FR2.3	If a study year does not meet the new curriculum requirements, it should open the old Keuzewijzer	Must

4.2.2.2. Use case description

Use Case	Select study year
Actor	Student
Precondition	The student has accessed the Keuzewijzer webpage.
Main Flow	1. The student opens the Keuzewijzer webpage. 2. The system displays a list of years for the student to select. 3. The student selects a study year from the list. 4. The system displays the overview of study paths available for the selected year.
Postcondition	The system displays the relevant paths for the chosen study year.
Alternative Flow	(3a) If the selected study year does not meet the new curriculum requirements, the system opens the old Keuzewijzer to show the available paths based on the previous curriculum.

4.2.2.3. Designs

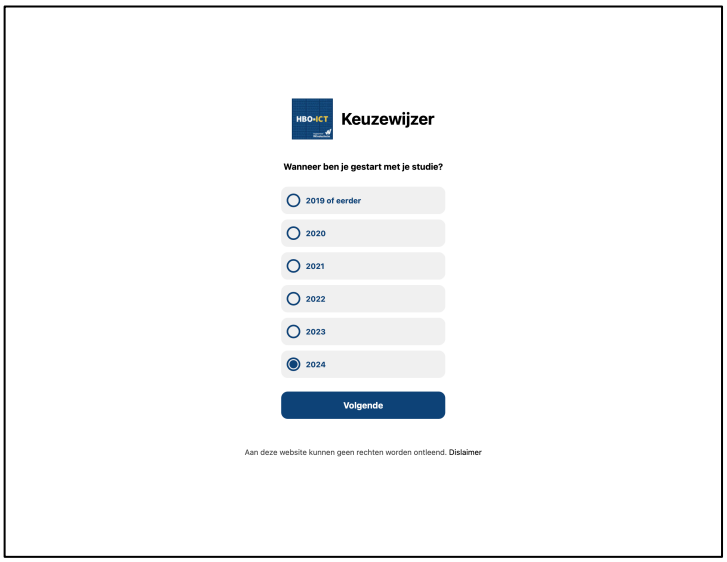


Figure 5 Tablet/Desktop design of selecting the study year

4.2.3. Select graduate profile

As a student, I want to select a graduation profile, so that I can see what clusters belong to it.

4.2.3.1. Acceptance criteria

#	Description	MoSCoW
FR3.1	When opening the webpage, it should show a list of graduate profiles that can be selected to view the path you have to take to graduate to that specific graduate profile.	Must
FR3.2	The current selected graduate profile can be changed.	Must
NFR1.2	Each graduate profile should have its own unique color.	Must

4.2.3.2. Use case description

Use Case	Select graduate profile
Actor	Student
Precondition	The student has picked a study year belonging to the new curriculum.
Main Flow	1. The system displays a list of graduate profiles for the student to choose from. 2. The student selects a graduate profile.
Postcondition	The system provides a view of the clusters belonging to the chose graduate profile.
Alternative Flow	None

4.2.3.3. Designs

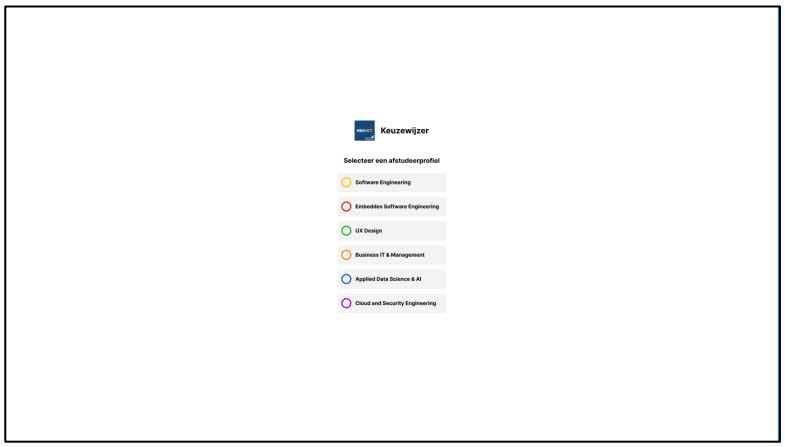


Figure 6 Tablet/desktop design of the graduate profiles use case

4.2.4. Select study route

As a student, I want to see a path from start to graduation, so I can track my progress and understand the steps needed to complete my degree.

4.2.4.1. Acceptance criteria

#	Description	MoSCoW
FR4.1	Show a path that shows the study year	Must
NFR1.3	The path matches the color of the selected graduate profile.	Must
FR4.2	Per study year there are 2 sections where a cluster can be followed.	Must
FR4.3	If there are multiple clusters available in a section, make sure that they are stacked and can be opened to select a cluster that a student likes.	Should
NFR1.4	When the last study year has been finished, it should show a graduate hat icon.	Should
FR4.4	Shows free choice if available.	Should
FR4.5	Propaedeutic should be fixed.	Must

4.2.4.2. Use case description

Use Case	Select study route
Actor	Student
Precondition	The student has picked a graduation profile.
Main Flow	1. The system displays a path with the required clusters belonging to the graduation profile. 2. The student chooses a cluster in the or section of the graduation profile. 3. The student picks their clusters for free choice.

Postcondition	The system displays the expected graduation direction.
Alternative Flow	None

4.2.4.3. Designs

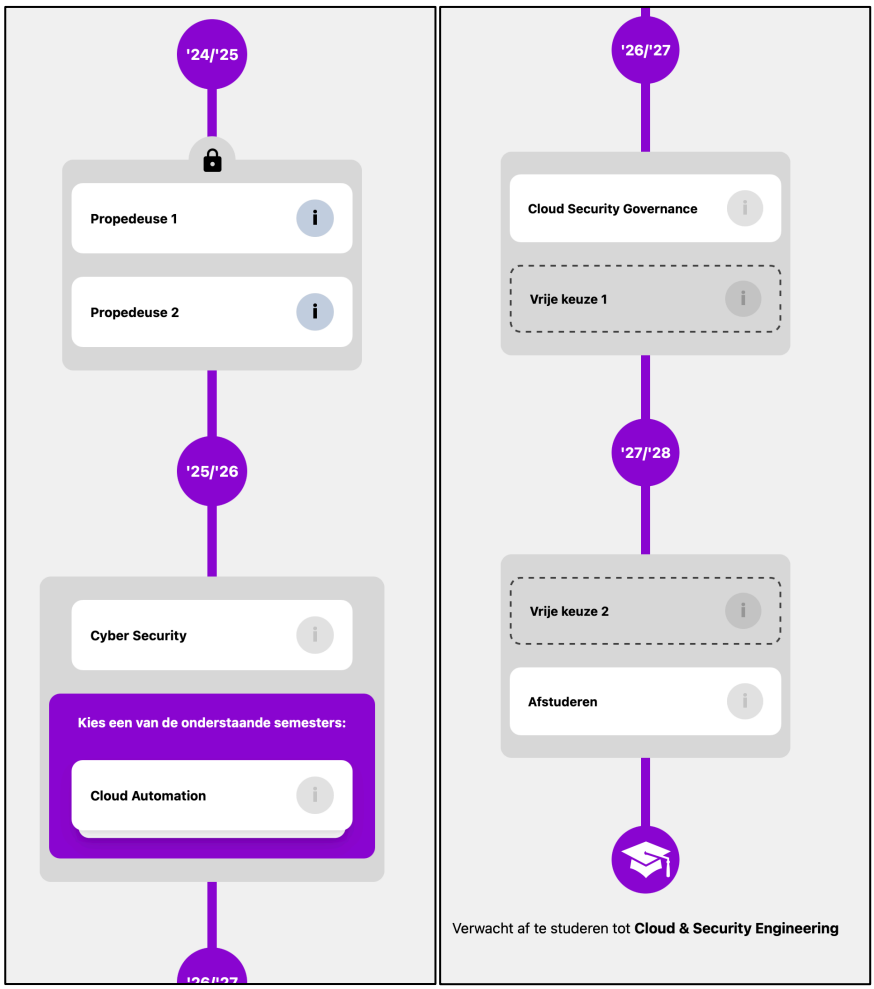


Figure 7 Study years design mobile and desktop

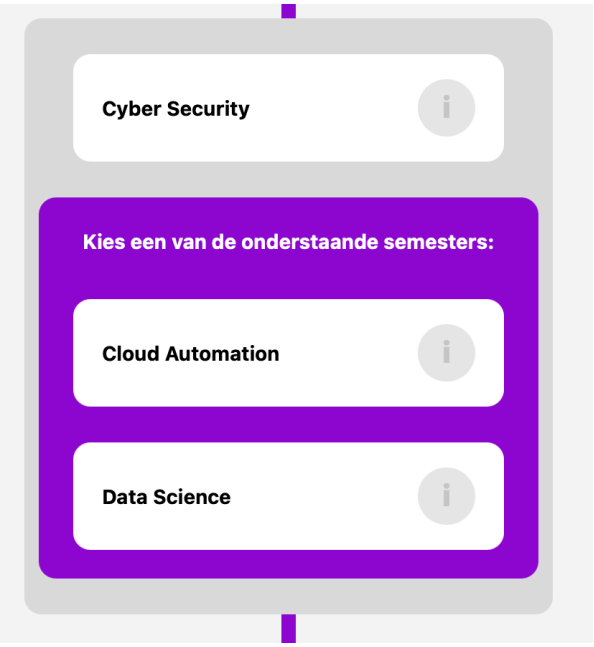


Figure 8 Choose for cluster (open)

4.2.5. View Cluster Information

As a Student, I want to be able to click on a cluster to view more information about it, so that I can better understand what I will learn in each cluster.

4.2.5.1. Acceptance criteria

#	Description	MoSCoW
FR5.1	If a link is available for the cluster, it should show a link button.	Must
FR5.2	A new window should be opened when clicking on the information.	Must
NFR1.5	If there is no link available for the cluster, it should show a grayed-out link button.	Should

4.2.5.2. Use case description

Use Case	View Cluster Information
Actor	Student
Precondition	The student has selected a graduation profile.
Main Flow	1. The student clicks on the info icon in the cluster they want information on.
Postcondition	The system navigates to the SharePoint page belonging to the cluster.
Alternative Flow	(1a) If cluster information is temporarily unavailable, the system notifies the student and suggests checking back later or contacting an advisor.

4.2.6. Insert free choice

As a Student, I want to be able to choose a cluster in the designated free choice cluster spaces, so that I can explore courses or topics that align with my personal interests and career goals.

4.2.6.1. Acceptance criteria

#	Description	MoSCoW
FR6.1	The system should allow students to select a cluster during the designated free choice year.	Must
FR6.2	The available clusters should be presented in a list.	Must
FR6.3	Students should be able to view prerequisites for each cluster before selection.	Should
FR6.4	The system should notify students if a selected cluster conflicts with program requirements.	Should
FR6.5	The selected choice should be able to be changed easily.	Should

4.2.6.2. Use case description

Use Case	Insert free choice
Actor	Student
Precondition	The student has reached the free choice year in their academic path.
Main Flow	1. The student clicks on the free choice selection. 2. The system displays available clusters. 3. The student selects a cluster from the list 4. The system saves the selection.
Postcondition	The path now has the selected cluster in it.
Alternative Flow	(3a) If the student clicks out without selecting a cluster it stays empty.

4.2.7. Retake cluster

As a student, I want to be able to retake a cluster, so that I can redo a cluster I failed.

4.2.7.1. Acceptance criteria

#	Description	MoSCoW
FR7.1	The system should allow students to retake a failed cluster.	Must
FR7.2	If the retaken cluster can be followed the semester after the failed cluster, it should be able to retake the failed cluster.	Must
FR7.3	If the retaken cluster can't be followed the semester after the failed cluster, that spot should become a free choice spot and the cluster to retake goes to the first possible spot where it can be taken.	Must
FR7.4	The system should notify students if it's not possible to retake a failed cluster after the failed cluster.	Should

4.2.7.2. Use case description

Use Case	Retake cluster
Actor	Student
Precondition	The student has failed a cluster
Main Flow	1. The student clicks on the change button 2. The system displays available clusters. 3. The student selects the same cluster to retake 4. The system saves the selection.
Postcondition	The path now has two times the same cluster to show it has retaken a cluster.
Alternative Flow	(3a) If the student clicks out without selecting the retaken cluster, it won't show that the student has retaken a cluster.

4.2.7.3. Designs

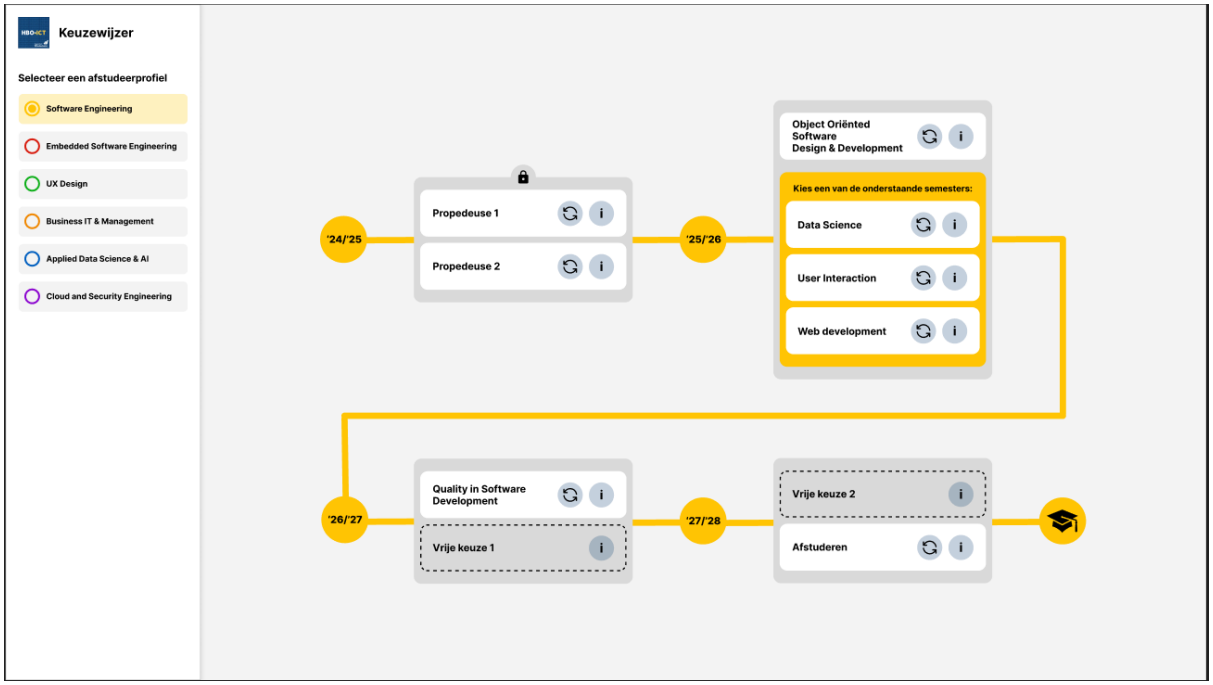


Figure 9 Tablet/Desktop design retake cluster

4.3. Dashboard

In this chapter all the user stories that belong to the dashboard of the Keuzewijzer will be described.

4.3.1. Login dashboard

As a maintainer, I want to login to the dashboard, so that I can manage clusters and perform other administrative tasks.

4.3.1.1. Acceptance criteria

#	Description	MoSCoW
FR8.1	There should be an input for the username/email of the administrative account	Must
FR8.2	There should be an input for the password	Must
FR8.3	There should be a button that lets you login	Must
FR8.4	Inputs should be validated for SQL or code injections	Must
FR8.5	A message should be shown for invalid credentials	Must
FR8.6	Login should only succeed if the username and password match stored credentials	Must
FR8.7	There should be a "Forgot password?" link to recover login details	Should
FR8.8	The system should lock the account after 5 failed login attempts	Should
FR8.9	After successful login, the user should be redirected to the dashboard	Must
FR8.10	Password should be stored securely using hashing/encryption techniques	Must
FR8.11	The session should expire after a set period of inactivity (e.g., 30 minutes)	Should
NFR1.6	The interface should display a loading indicator while the login request is processed	Could
FR8.12	Login process should work across all supported browsers and devices	Should

4.3.1.2. Use case description

Use Case	Login dashboard
Actor	Maintainer
Precondition	The maintainer has accessed the Dashboard login page.
Main Flow	1. The maintainer opens the Dashboard login page. 2. The system displays input fields for the maintainer to enter their username and password.

	3. The maintainer enters their credentials and submits them. 4. The system verifies the credentials. 5. Upon successful verification, the system grants access to the maintainer dashboard.
Postcondition	The maintainer is logged in and has access to administrative features in the Dashboard.
Alternative Flow	(3a) If the maintainer enters incorrect credentials, the system displays an error message and prompts for re-entry.

4.3.2. Reset password

As a maintainer, I want to reset my password if I forget it, so that I can regain access to the dashboard.

4.3.2.1. Acceptance criteria

#	Description	MoSCoW
FR9.1	There should be a "Forgot Password?" link on the login page.	Must
FR9.2	The link should direct the user to a password recovery form.	Must
FR9.3	There should be an input field for entering the username.	Must
FR9.4	An email should be sent with a secure token for password reset.	Must
FR9.5	The token should expire after a predefined period (e.g., 30 mins).	Must
FR9.6	If the user enters an invalid email, an error message should appear.	Must
FR9.7	The system should validate the token before allowing a reset.	Must
FR9.8	There should be input fields to enter and confirm the new password.	Must
FR9.9	The new password should follow defines complexity rules.	Should
FR9.10	A success message should be shown after a successful reset.	Should

4.3.2.2. Use case description

Use Case	Reset password
Actor	Maintainer
Precondition	The maintainer has accessed the forgot password page.
Main Flow	1. The maintainer clicks on the "Forgot Password?" link on the login page. 2. The system displays a form to enter the registered username/email. 3. The maintainer enters their email and submits the form.

	4. The system sends a secure token to the registered email address. 5. The maintainer clicks the link in the email and is redirected to the password reset page. 6. The maintainer enters and confirms the new password. 7. The system verifies the token and updates the password.
Postcondition	The maintainer can log in with the new password.
Alternative Flow	(3a): If the email address does not exist, the system displays an error message. (6a): If the token is expired or invalid, the system prompts the maintainer to request a new one

4.3.3. Manage graduate profiles

As a maintainer, I want to manage graduate profiles, so that I can update relevant information efficiently.

4.3.3.1. Acceptance criteria

#	Description	MoSCoW
FR10.1	The system should display a list of graduate profiles.	Must
FR10.2	The maintainer can add a new graduate profile via a form.	Must
FR10.3	The maintainer can edit existing graduate profiles.	Must
FR10.4	The maintainer can delete a profile with confirmation.	Must
FR10.5	The system should validate data before saving changes.	Should
FR10.6	All operations should be logged for auditing purposes.	Should

4.3.3.2. Use case description

Use Case	Manage graduate profiles
Actor	Maintainer
Precondition	The maintainer has access to the dashboard.
Main Flow	1. The maintainer accesses the graduate profiles section in the dashboard. 2. The system displays a list of graduate profiles in a table format. 3. The maintainer selects an action: add, edit, or delete. 4. The system validates the input and processes the request.
Postcondition	The graduate profile list is updated with the maintainer's changes.

Alternative Flow	(4a): If the input data is invalid, the system displays an error message and prompts for corrections. (4b): If the maintainer cancels an action, the system returns to the list view without making changes.

4.3.4. Manage cohorts

As a maintainer, I want to manage cohorts, so that I can update relevant information efficiently.

4.3.4.1. Acceptance criteria

#	Description	MoSCoW
FR11.1	The system should display a list of cohorts	Must
FR11.2	The maintainer can add a new cohort via a form.	Must
FR11.3	The maintainer can edit existing cohorts.	Must
FR11.4	The maintainer can delete a cohort with confirmation.	Must
FR11.5	The system should validate data before saving changes.	Should
FR11.6	All operations should be logged for auditing purposes.	Should

4.3.4.2. Use case description

Use Case	Manage cohorts
Actor	Maintainer
Precondition	The maintainer has access to the dashboard.
Main Flow	1. The maintainer accesses the cohorts section in the dashboard. 2. The system displays a list of cohorts in a table format. 3. The maintainer selects an action: add, edit, or delete. 4. The system validates the input and processes the request.
Postcondition	The cohorts list is updated with the maintainer's changes.
Alternative Flow	(4a): If the input data is invalid, the system displays an error message and prompts for corrections. (4b): If the maintainer cancels an action, the system returns to the list view without making changes.

4.3.5. Manage clusters

As a maintainer, I want to manage clusters, so that I can organize related graduate profiles efficiently.

4.3.5.1. Acceptance criteria

#	Description	MoSCoW
FR12.1	The system should display a list of clusters.	Must
FR12.2	The maintainer can add a new cluster via a form.	Must
FR12.3	The maintainer can edit existing clusters.	Must
FR12.4	The maintainer can delete a cluster with confirmation.	Must
FR12.5	The maintainer can associate clusters with graduate profiles.	Must
FR12.6	The system should validate data before saving changes.	Should
FR12.7	All operations should be logged for auditing purposes.	Should

4.3.5.2. Use case description

Use Case	Manage Clusters
Actor	Maintainer
Precondition	The maintainer has access to the dashboard.
Main Flow	1. The maintainer accesses the clusters section in the dashboard. 2. The system displays a list of clusters in a table format. 3. The maintainer selects an action: add, edit, or delete. 4. The maintainer can associate or dissociate clusters with graduate profiles. 5. The system validates the input and processes the request.
Postcondition	The cluster list is updated with the maintainer's changes.
Alternative Flow	(4a): If the input data is invalid, the system displays an error message and prompts for corrections. (4b): If the maintainer cancels an action, the system returns to the list view without making changes.

4.3.6. Manage users

As a maintainer, I want to manage users, so that I can control system access and roles effectively.

4.3.6.1. Acceptance criteria

#	Description	MoSCoW
FR13.1	The system should display a list of users.	Must
FR13.2	The maintainer can add a new user via a form.	Must
FR13.3	The maintainer can edit existing user information.	Must
FR13.4	The maintainer can delete a user with confirmation.	Must
FR13.5	The system should allow assigning roles to users.	Could
FR13.6	The system should validate data before saving changes.	Should
FR13.7	All operations should be logged for auditing purposes.	Should

4.3.6.2. Use case description

Use Case	Manage users
Actor	Maintainer
Precondition	The maintainer has access to the dashboard.
Main Flow	1. The maintainer accesses the users section in the dashboard. 2. The system displays a list of users in a table format. 3. The maintainer selects an action: add, edit, or delete. 4. The maintainer can assign or update roles for a user. 5. The system validates the input and processes the request.
Postcondition	The user list is updated with the maintainer's changes.
Alternative Flow	(4a): If the input data is invalid, the system displays an error message and prompts for corrections. (4b): If the maintainer cancels an action, the system returns to the list view without making changes.

4.4. General

In this chapter general user stories, which belong to both the dashboard and the Keuzewijzer.

4.4.1. Implementing a CI/CD pipeline

As a Product Owner, I want to establish a Continuous Integration and Continuous Deployment (CI/CD) pipeline, so that we can automate our build, testing, and deployment processes to improve code quality and accelerate the delivery of features.

4.4.1.1. Acceptance criteria

Description	MoSCoW
The CI/CD pipeline is set up using appropriate tools (e.g., Jenkins, GitLab CI, CircleCI) that align with our project needs.	Must
The pipeline includes automated build processes that trigger on code commits to the repository.	Must
Automated testing (unit tests, integration tests) is integrated into the pipeline to ensure code quality before deployment.	Must
The deployment process to staging and production environments is automated, reducing manual steps and potential errors.	Must
Notifications are configured to alert the team of build and deployment statuses, including failures and successes.	Must
Documentation for the CI/CD pipeline is created, including setup instructions, workflows, and troubleshooting guidelines.	Should
The pipeline is regularly reviewed and optimized based on team feedback and project needs.	Should

Figures

Figure 1 Domain model..... 2
Figure 2 Use Case Diagram of Keuzewijzer and Dashboard..... 3
Figure 3 Mobile design Graduate profiles. 7
Figure 4 Tablet/Desktop design of location graduate profiles 7
Figure 5 Tablet/Desktop design of selecting the study year 9
Figure 6 Tablet/desktop design of the graduate profiles use case..... 10
Figure 7 Study years design mobile and desktop 11
Figure 8 Choose for cluster (open)..... 12